tapat

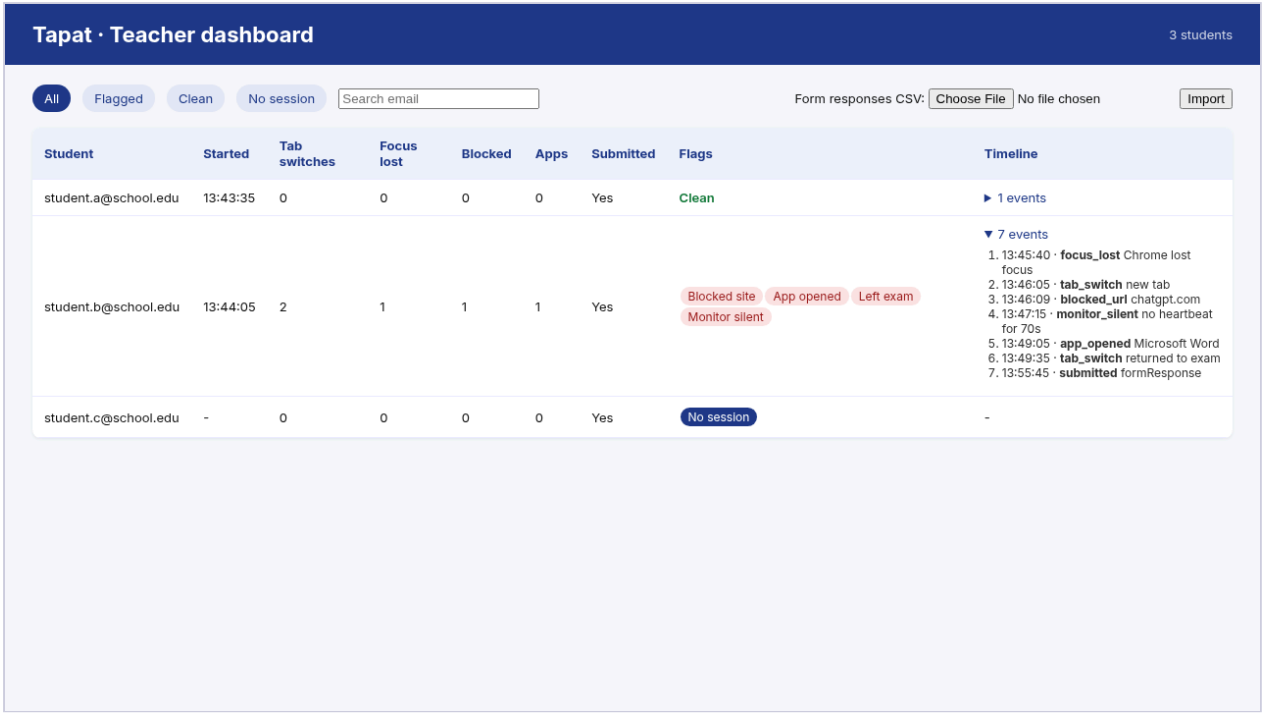
An exam monitor for Google Forms quizzes that blocks a listed set of sites and logs when students leave the exam.

INTE301 Systems Integration & Architecture · Project 1

Sir Rodel Villareaz

Troy Lazaro

October 12, 2026



Contents

1. Overview
2. The problem
3. Features
4. Architecture
5. Setup
6. Usage
7. Screenshots
8. Limitations
9. Team
10. Code listings
    a. Teacher server
    b. Chrome extension
    c. Desktop watcher
    d. Documentation build



Tapat is an exam monitor for Google Forms quizzes. It blocks a listed set of sites while the quiz is open, logs when a student leaves the exam, and gives the teacher one dashboard to review every student afterward.

Tapat is Filipino for "honest."

1. Overview

A student opens the quiz through the Tapat Chrome extension. The extension shows a consent screen, embeds the Google Form, blocks a listed set of sites, and logs tab switches and Chrome losing focus. An optional desktop watcher logs when Word, Notes, Preview or Acrobat comes to the front. Every event goes to a small teacher server, which shows one row per student on a dashboard. After the quiz, the teacher imports the Form's responses CSV, and anyone who submitted without Tapat running gets a **No session** flag.

2. The problem

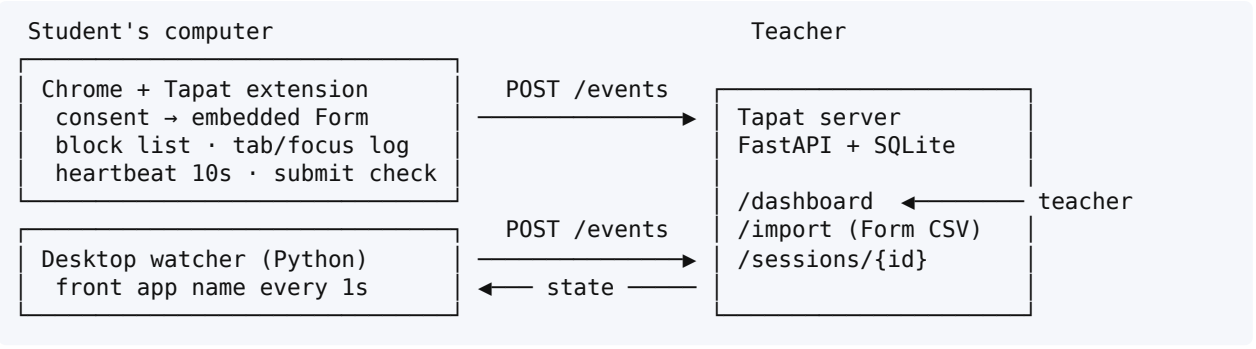
Online quizzes on Google Forms are easy to take with a second tab open. A student can ask ChatGPT, open their notes in Word, or read a PDF of the lesson, and the teacher sees only the final score. Project 1 asks for a system that addresses Google Forms quizzes being answered while opening webpages, ChatGPT, Word, notes and PDFs.

3. Features

- **Consent first.** Before the quiz starts, the student sees: *"This logs tab switches and apps opened, nothing else."*
- **Site blocking.** Blocks a listed set of sites during the quiz: ChatGPT, Gemini, Claude, Word Online / office.com, Google Docs and any `.pdf` link. Blocked attempts are logged.
- **Leaving the exam.** Logs tab switches, new tabs and Chrome losing focus.
- **Desktop apps.** The watcher logs app names only, never window titles, and exits on its own when the Form is submitted.
- **Submit detection.** The session closes when the embedded Form reaches its `formResponse` page.
- **Heartbeat.** The extension checks in every 10 seconds. A gap over 25 seconds raises a **Monitor silent** flag.
- **No session flag.** For students who submitted the Form without Tapat running, found by importing the Form's responses CSV.
- **Teacher dashboard.** One row per student (by email), flag counts, a timeline of events, and filters for All, Flagged, Clean and No session.

4. Architecture

Part	Folder	Built with	Job
Chrome extension	`extension/`	Manifest V3, JavaScript	Consent, Form embed, blocking, tab/focus logging, submit detection, heartbeat
Desktop watcher	`watcher/`	Python	Logs which watched app is in front, by name only
Teacher server	`server/`	FastAPI, SQLite	Stores events, imports the Form CSV, serves the dashboard



Every event has the same shape:

```
{"session_id": "…", "student": "email", "type": "tab_switch | focus_lost | blocked_url |
app_opened | heartbeat | submitted", "detail": "…", "ts": 1760250000.0}
```

5. Setup

1. **In the Google Form, turn on Collect email addresses.** Tapat matches students by email (Settings → Responses → Collect email addresses).
2. **Start the server** (Python 3.10+): `cd server ./run.sh --seed # --seed loads three sample students` The dashboard is at http://localhost:8000/dashboard.
3. **Load the extension:** open `chrome://extensions`, turn on Developer mode, click **Load unpacked** and pick the `extension/` folder.
4. **Optional, the watcher:** `python watcher/watcher.py --email you@school.edu --session <id shown on the exam page>`. On Windows, first run `pip install -r watcher/requirements.txt`.

6. Usage

Student 1. Click the Tapat icon, enter your school email and paste the Form link. 2. Read the consent screen and start the quiz. 3. Answer and submit the Form as usual. Tapat stops when the response is recorded.

Teacher 1. Open the dashboard while the quiz runs to see each student's flags and timeline. 2. After the quiz, download the Form's responses as CSV and import it on the dashboard. 3. Filter

by **Flagged** or **No session** to see who to follow up with.

Run the tests with `cd server && python -m pytest tests`.

7. Screenshots

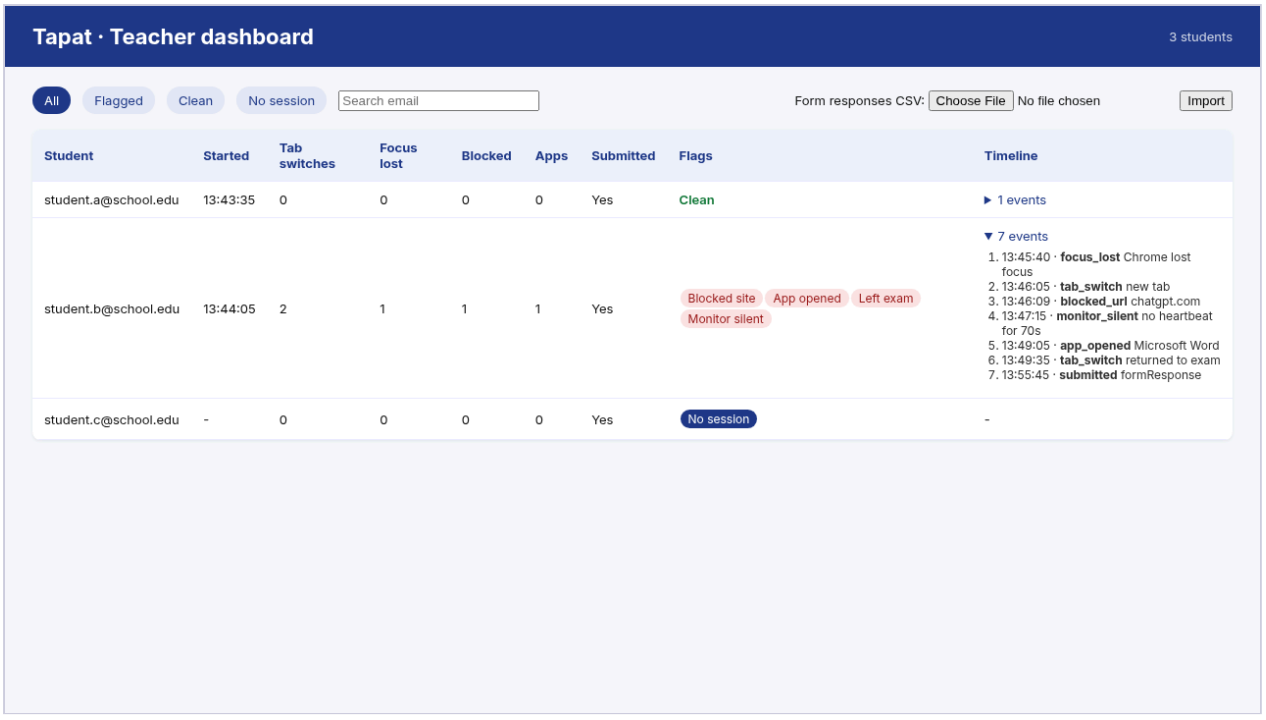


Figure 1. Teacher dashboard with seed data: a clean student, a flagged student with an open timeline, and a No session row.

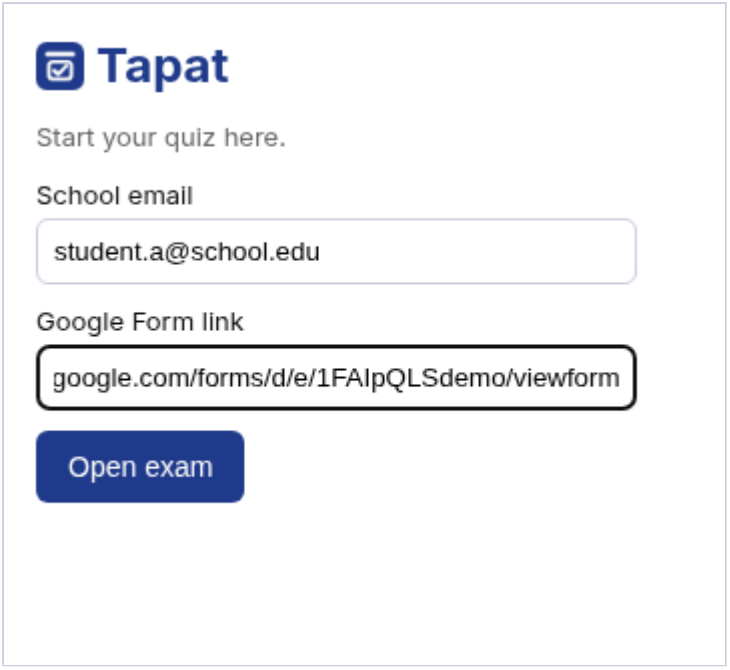


Figure 2. Extension popup where the student enters their email and the Form link.

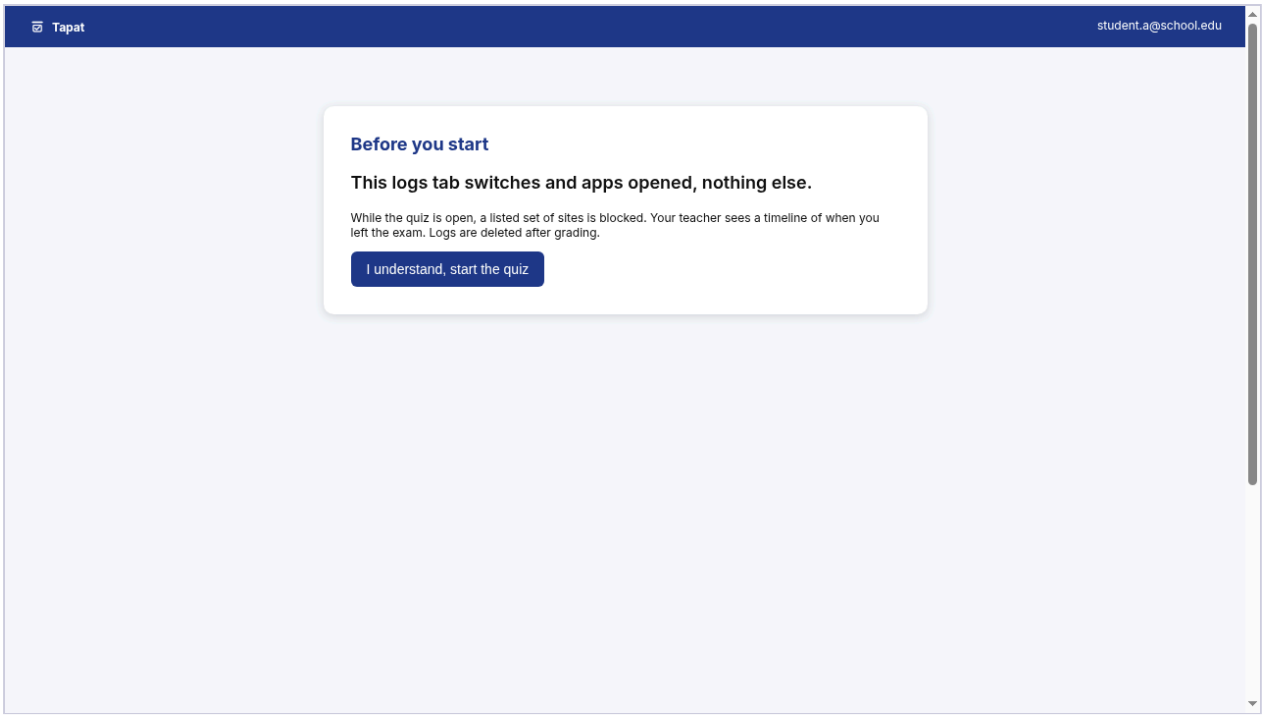


Figure 3. Consent screen shown before the quiz starts.

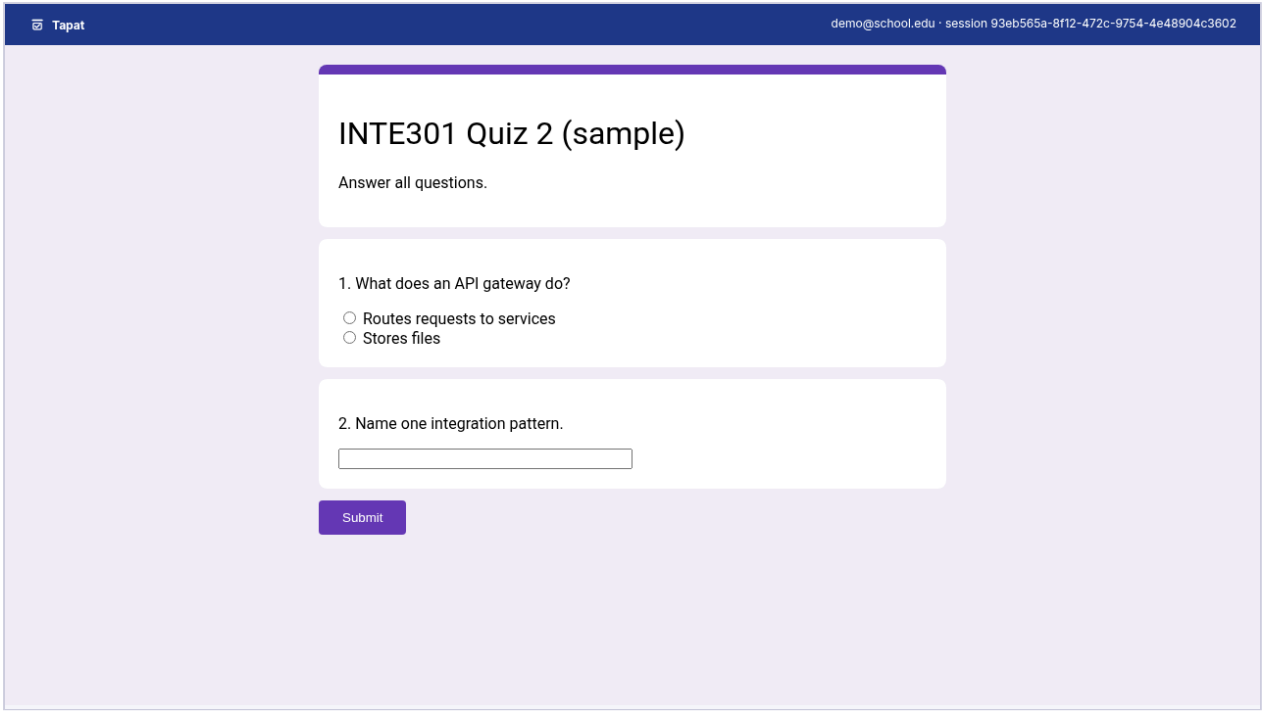


Figure 4. Exam page with a sample quiz embedded in place of a real Google Form.

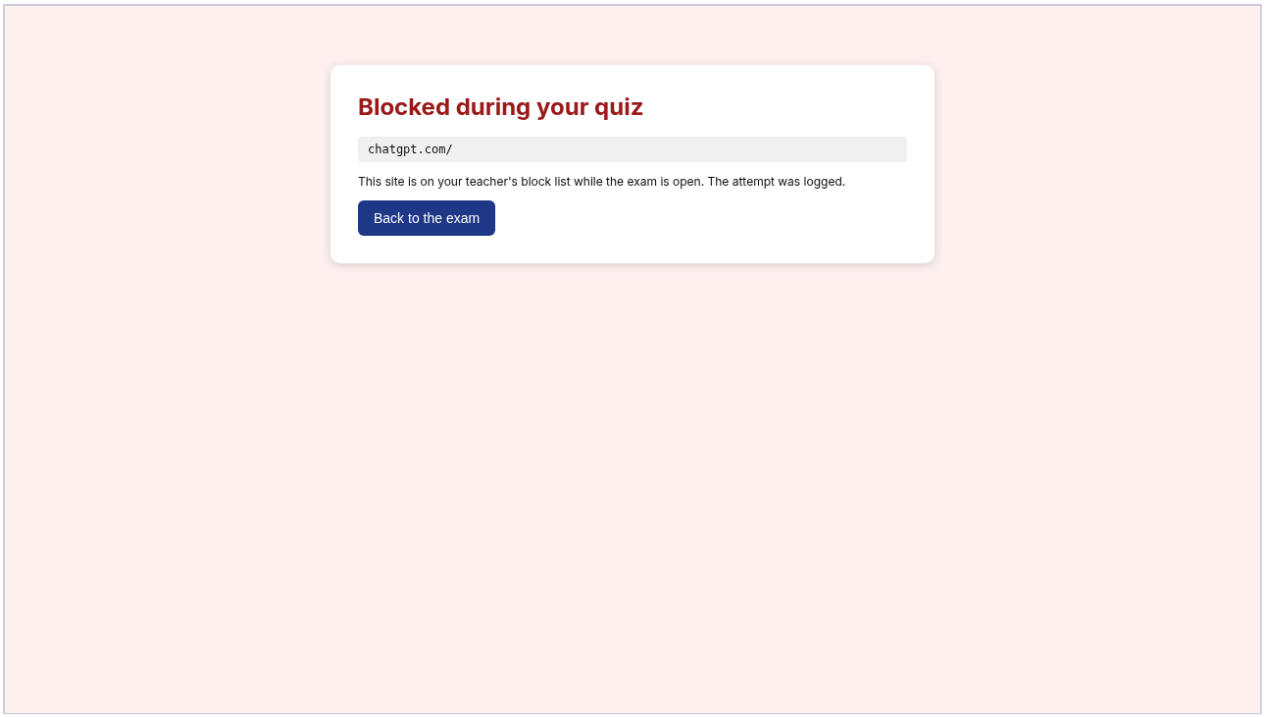


Figure 5. Block page shown when chatgpt.com is opened during the quiz; the attempt is logged.

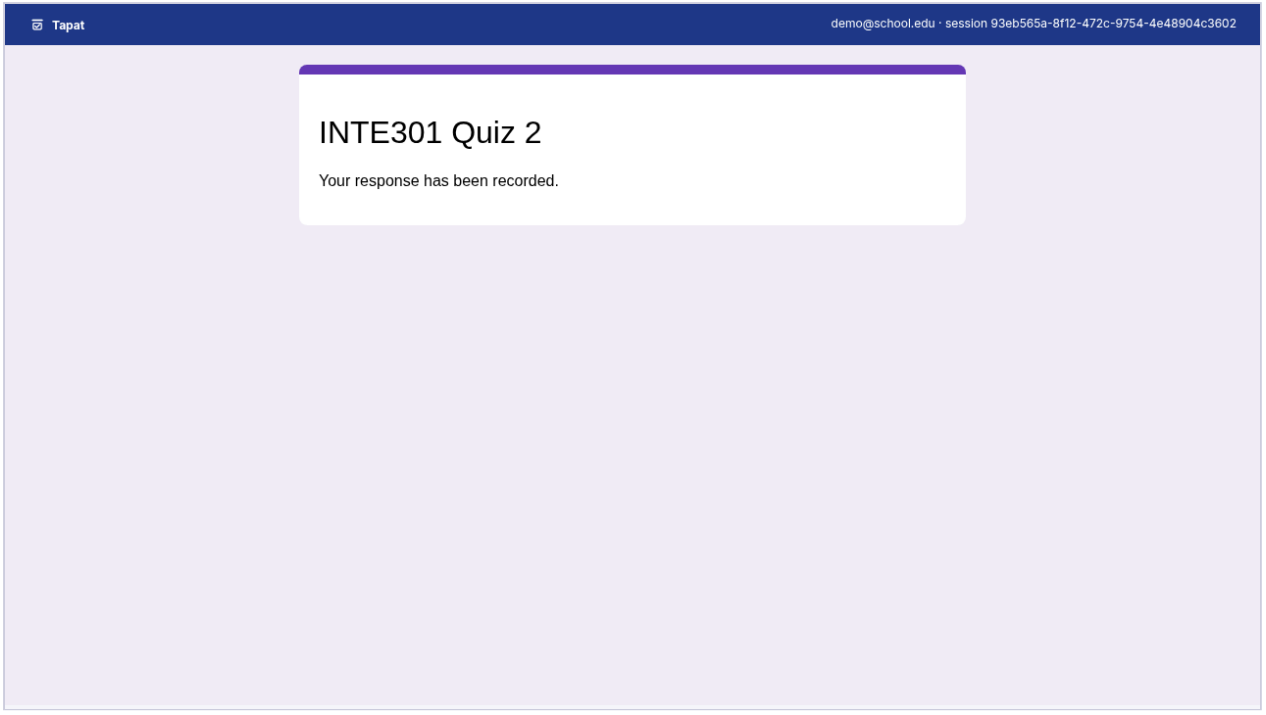


Figure 6. Sample quiz submitted: Tapat detects the formResponse page and closes the session.

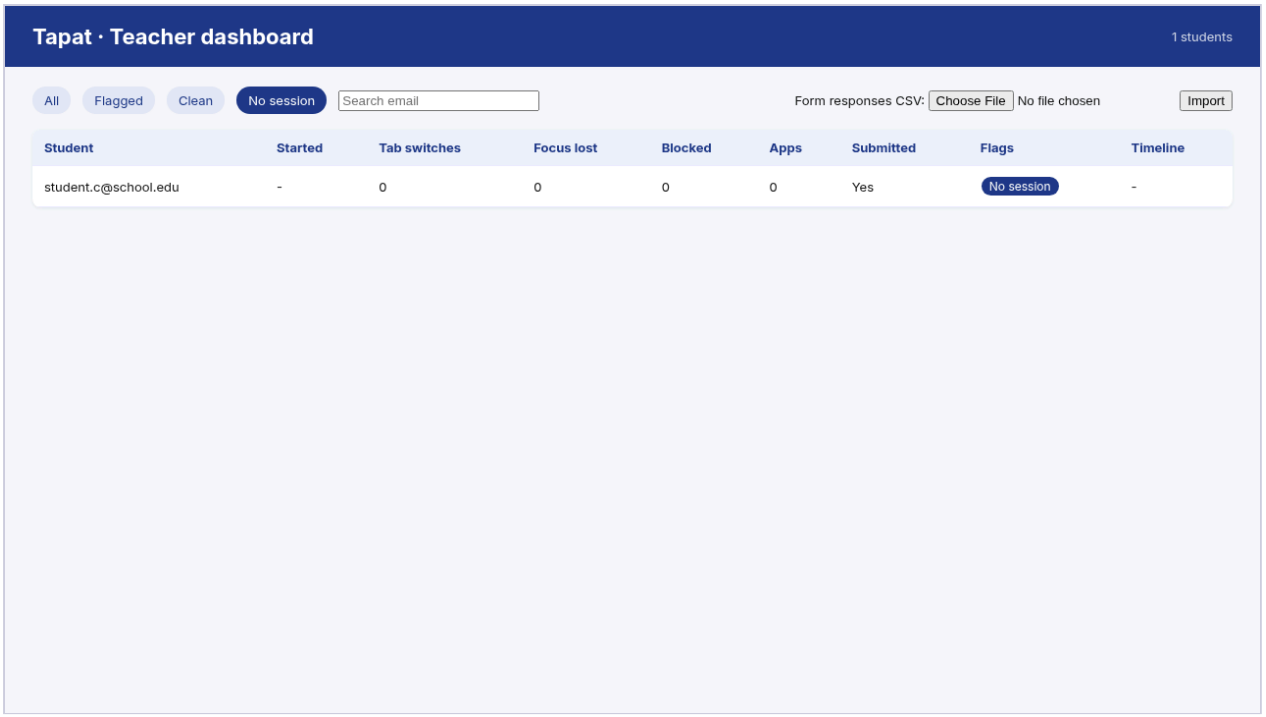


Figure 7. Submitted in incognito without Tapat, flagged after CSV import.

8. Limitations

- It blocks a list, not all AI. A site that isn't on the list stays open, and a second device can't be seen at all.
- The watcher only runs if the student starts it, and it exits on submit.
- Students who decline take the quiz on paper or in a proctored room.
- A flag is a reason to ask, not proof of cheating. The teacher decides.
- Logs are deleted after grading. Tapat stores emails and event types only, never page contents, keystrokes or window titles.

9. Team

- Troy Lazaro

Code listing: Teacher server

6 files.

server/app.py · Python · 199 lines

```python
"""Tapat teacher server: receives monitor events and shows the dashboard."""
import csv
import io
import os
import sqlite3
import time
from datetime import datetime, timezone
from html import escape

from fastapi import FastAPI, File, Query, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import HTMLResponse, RedirectResponse
from pydantic import BaseModel

DB_PATH = os.environ.get("TAPAT_DB", os.path.join(os.path.dirname(__file__), "tapat.db"))
SILENT_GAP = 25  # seconds without a heartbeat before "Monitor silent"
EVENT_TYPES = {"tab_switch", "focus_lost", "blocked_url", "app_opened", "heartbeat", "submitted"}

app = FastAPI(title="Tapat")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])


def db():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    conn.execute(
        """CREATE TABLE IF NOT EXISTS events(
            id INTEGER PRIMARY KEY, session_id TEXT, student TEXT,
            type TEXT, detail TEXT, ts REAL)"""
    )
    conn.execute(
        """CREATE TABLE IF NOT EXISTS responses(
            email TEXT PRIMARY KEY, submitted_at TEXT)"""
    )
    return conn


class Event(BaseModel):
    session_id: str
    student: str
    type: str
    detail: str = ""
    ts: float | None = None


@app.post("/events")
def post_event(ev: Event):
    if ev.type not in EVENT_TYPES:
        return {"ok": False, "error": f"unknown type {ev.type}"}
    with db() as conn:
        conn.execute(
            "INSERT INTO events(session_id, student, type, detail, ts) VALUES (?,?,?,?,?)",
            (ev.session_id, ev.student.strip().lower(), ev.type, ev.detail[:300], ev.ts or time.time()),
        )
    return {"ok": True}
```

```
 56
 57
 58  @app.get("/sessions/{session_id}")
 59  def session_state(session_id: str):
 60      """Polled by the desktop watcher; 'closed' once the Form is submitted."""
 61      with db() as conn:
 62          row = conn.execute(
 63              "SELECT 1 FROM events WHERE session_id=? AND type='submitted' LIMIT 1",
(session_id,)
 64          ).fetchone()
 65          known = conn.execute("SELECT 1 FROM events WHERE session_id=? LIMIT 1", (se
ssion_id,)).fetchone()
 66      return {"session_id": session_id, "state": "closed" if row else ("open" if know
n else "unknown")}
 67
 68
 69  @app.post("/import")
 70  async def import_csv(file: UploadFile = File(...)):
 71      """Import the Google Form responses CSV. Needs 'Collect email addresses' on."""
 72      text = (await file.read()).decode("utf-8-sig")
 73      count = import_rows(text)
 74      return RedirectResponse(f"/dashboard?imported={count}", status_code=303)
 75
 76
 77  def import_rows(text: str) -> int:
 78      reader = csv.DictReader(io.StringIO(text))
 79      email_col = next((c for c in (reader.fieldnames or []) if "email" in c.lower
()), None)
 80      if not email_col:
 81          return 0
 82      ts_col = next((c for c in reader.fieldnames if "timestamp" in c.lower()), None)
 83      n = 0
 84      with db() as conn:
 85          for r in reader:
 86              email = (r.get(email_col) or "").strip().lower()
 87              if email:
 88                  conn.execute(
 89                      "INSERT OR REPLACE INTO responses(email, submitted_at) VALUES
(?,?)",
 90                      (email, r.get(ts_col, "") if ts_col else ""),
 91                  )
 92                  n += 1
 93      return n
 94
 95
 96  def summarize(now: float | None = None):
 97      now = now or time.time()
 98      with db() as conn:
 99          events = conn.execute("SELECT * FROM events ORDER BY ts").fetchall()
100          responses = {r["email"]: r["submitted_at"] for r in conn.execute("SELECT *
FROM responses")}
101      students: dict[str, dict] = {}
102      for e in events:
103          s = students.setdefault(e["student"], {
104              "email": e["student"], "counts": {t: 0 for t in EVENT_TYPES}, "timelin
e": [],
105              "last_beat": None, "first": e["ts"], "submitted": False, "silent_gaps":
0,
106          })
107          if e["type"] == "heartbeat":
108              if s["last_beat"] and e["ts"] - s["last_beat"] > SILENT_GAP:
109                  s["silent_gaps"] += 1
110                  s["timeline"].append((s["last_beat"], "monitor_silent", f"no heartb
```

```
eat for {int(e['ts'] - s['last_beat'])}s"))
            s["last_beat"] = e["ts"]
        else:
            s["timeline"].append((e["ts"], e["type"], e["detail"]))
        s["counts"][e["type"]] += 1
        if e["type"] == "submitted":
            s["submitted"] = True
    rows = []
    for s in students.values():
        flags = []
        if s["counts"]["blocked_url"]:
            flags.append("Blocked site")
        if s["counts"]["app_opened"]:
            flags.append("App opened")
        if s["counts"]["tab_switch"] + s["counts"]["focus_lost"] >= 3:
            flags.append("Left exam")
        live_gap = (not s["submitted"]) and s["last_beat"] and now - s["last_beat"]
> SILENT_GAP
        if s["silent_gaps"] or live_gap:
            flags.append("Monitor silent")
        s["flags"] = flags
        s["in_form"] = s["email"] in responses
        rows.append(s)
    for email, at in responses.items():
        if email not in students:
            rows.append({"email": email, "counts": {t: 0 for t in EVENT_TYPES}, "ti
meline": [],
                         "flags": ["No session"], "submitted": True, "in_form": Tru
e, "first": None})
    return sorted(rows, key=lambda r: r["email"])


def fmt(ts):
    return datetime.fromtimestamp(ts).strftime("%H:%M:%S") if ts else "-"


CSS = """
body{font-family:Inter,Segoe UI,Helvetica,Arial,sans-serif;margin:0;background:#f5f
7fb;color:#111}
header{background:#1F3A8A;color:#fff;padding:18px 28px;display:flex;justify-conten
t:space-between;align-items:center}
header h1{margin:0;font-size:22px}header span{opacity:.8;font-size:13px}
main{padding:20px 28px}
.bar{display:flex;gap:12px;align-items:center;margin-bottom:16px;flex-wrap:wrap}
.bar a{padding:6px 12px;border-radius:16px;background:#e3e8f5;color:#1F3A8A;text-de
coration:none;font-size:13px}
.bar a.on{background:#1F3A8A;color:#fff}
form.imp{margin-left:auto;font-size:13px}
table{width:100%;border-collapse:collapse;background:#fff;border-radius:8px;overflo
w:hidden;box-shadow:0 1px 3px #0001}
th,td{padding:10px 12px;text-align:left;font-size:13px;border-bottom:1px solid #ee
f}
th{background:#eef2fb;color:#1F3A8A}
.flag{display:inline-block;padding:2px 8px;border-radius:10px;font-size:12px;margi
n:1px;background:#fde2e2;color:#9b1c1c}
.flag.No{background:#1F3A8A;color:#fff}.clean{color:#127a3a;font-weight:600}
details summary{cursor:pointer;color:#1F3A8A}
ol.tl{margin:6px 0;padding-left:18px;font-size:12px;color:#333}
"""


@app.get("/", include_in_schema=False)
def root():
```

```
164      return RedirectResponse("/dashboard")
165
166
167  @app.get("/dashboard", response_class=HTMLResponse)
168  def dashboard(flag: str = Query("all"), q: str = Query(""), imported: int | None =
None):
169      rows = summarize()
170      if q:
171          rows = [r for r in rows if q.lower() in r["email"]]
172      if flag == "flagged":
173          rows = [r for r in rows if r["flags"]]
174      elif flag == "clean":
175          rows = [r for r in rows if not r["flags"]]
176      elif flag == "nosession":
177          rows = [r for r in rows if "No session" in r["flags"]]
178      tabs = "".join(
179          f'<a class="{"on" if flag == k else ""}" href="/dashboard?flag={k}">{v}</a
>'
180          for k, v in [("all", "All"), ("flagged", "Flagged"), ("clean", "Clean"),
("nosession", "No session")]
181      )
182      body = []
183      for r in rows:
184          c = r["counts"]
185          flags = "".join(f'<span class="flag {escape(f.split()[0])}">{escape(f)}</sp
an>' for f in r["flags"]) or '<span class="clean">Clean</span>'
186          tl = "".join(f"<li>{fmt(t)} · <b>{escape(k)}</b> {escape(d)}</li>" for t,
k, d in r["timeline"])
187          tl_html = f"<details><summary>{len(r['timeline'])} events</summary><ol clas
s='tl'>{tl}</ol></details>" if tl else "-"
188          body.append(
189              f"<tr><td>{escape(r['email'])}</td><td>{fmt(r['first'])}</td><td>{c['ta
b_switch']}</td><td>{c['focus_lost']}</td>"
190              f"<td>{c['blocked_url']}</td><td>{c['app_opened']}</td><td>{'Yes' if r
['submitted'] else 'No'}</td>"
191              f"<td>{flags}</td><td>{tl_html}</td></tr>"
192          )
193      note = f"<p>Imported {imported} Form responses.</p>" if imported is not None el
se ""
194      return f"""<!doctype html><html><head><meta charset=utf-8><title>Tapat dashboar
d</title><style>{CSS}</style></head>
195  <body><header><h1>Tapat · Teacher dashboard</h1><span>{len(rows)} students</span></
header><main>{note}
196  <div class="bar">{tabs}<form method=get action=/dashboard><input name=q placeholder
="Search email" value="{escape(q)}"></form>
197  <form class=imp method=post action=/import enctype=multipart/form-data>Form respons
es CSV: <input type=file name=file accept=.csv> <button>Import</button></form></div>
198  <table><tr><th>Student</th><th>Started</th><th>Tab switches</th><th>Focus lost</th>
<th>Blocked</th><th>Apps</th><th>Submitted</th><th>Flags</th><th>Timeline</th></tr>
199  {''.join(body) or '<tr><td colspan=9>No sessions yet.</td></tr>'}</table></main></b
ody></html>"""
```

server/seed.py · Python · 35 lines

```python
"""Fill the database with three sample students for demos and screenshots."""
import os
import time

from app import EVENT_TYPES, db, import_rows

def main():
    with db() as conn:
        conn.execute("DELETE FROM events")
        conn.execute("DELETE FROM responses")
        t0 = time.time() - 1200
        rows = []
        # Student A: clean run
        for i in range(0, 600, 10):
            rows.append(("s-a", "student.a@school.edu", "heartbeat", "", t0 + i))
        rows.append(("s-a", "student.a@school.edu", "submitted", "formResponse", t0 + 600))
        # Student B: tabs, ChatGPT block, Word, silent gap
        b = t0 + 30
        for i in list(range(0, 200, 10)) + list(range(260, 700, 10)):
            rows.append(("s-b", "student.b@school.edu", "heartbeat", "", b + i))
        rows += [
            ("s-b", "student.b@school.edu", "focus_lost", "Chrome lost focus", b + 95),
            ("s-b", "student.b@school.edu", "tab_switch", "new tab", b + 120),
            ("s-b", "student.b@school.edu", "blocked_url", "chatgpt.com", b + 124),
            ("s-b", "student.b@school.edu", "app_opened", "Microsoft Word", b + 300),
            ("s-b", "student.b@school.edu", "tab_switch", "returned to exam", b + 330),
            ("s-b", "student.b@school.edu", "submitted", "formResponse", b + 700),
        ]
        conn.executemany("INSERT INTO events(session_id, student, type, detail, ts) VALUES (?,?,?,?,?)", rows)
    # Student C: in the Form CSV but never ran Tapat (incognito)
    import_rows(open(os.path.join(os.path.dirname(__file__), "sample_responses.csv")).read())
    print("seeded")

if __name__ == "__main__":
    main()
```

server/run.sh · Bash · 7 lines

```bash
#!/usr/bin/env bash
# Start the Tapat server on http://localhost:8000 (add --seed for demo data)
set -e
cd "$(dirname "$0")"
[ -d .venv ] || { python3 -m venv .venv && .venv/bin/pip install -q -r requirements.t
xt; }
[ "$1" = "--seed" ] && .venv/bin/python seed.py
exec .venv/bin/uvicorn app:app --host 127.0.0.1 --port 8000
```

server/requirements.txt · Text · 5 lines

```
fastapi
uvicorn
python-multipart
pytest
httpx
```

server/sample_responses.csv · CSV · 4 lines

```
Timestamp,Email Address,Score
10/12/2026 14:10:02,student.a@school.edu,18 / 20
10/12/2026 14:12:40,student.b@school.edu,17 / 20
10/12/2026 14:09:11,student.c@school.edu,20 / 20
```

server/tests/test_app.py · Python · 42 lines

```python
import os, sys, time
sys.path.insert(0, os.path.dirname(os.path.dirname(__file__)))
os.environ["TAPAT_DB"] = "/tmp/tapat_test.db"
if os.path.exists("/tmp/tapat_test.db"):
    os.remove("/tmp/tapat_test.db")
from fastapi.testclient import TestClient
import app as tapat

c = TestClient(tapat.app)

def ev(t, detail="", ts=None, sid="x1", who="a@x.edu"):
    return c.post("/events", json={"session_id": sid, "student": who, "type": t, "detail": detail, "ts": ts}).json()

def test_post_and_dashboard():
    assert ev("blocked_url", "chatgpt.com")["ok"]
    html = c.get("/dashboard").text
    assert "a@x.edu" in html and "Blocked site" in html

def test_rejects_unknown_type():
    assert not ev("bogus")["ok"]

def test_session_closes_on_submit():
    assert c.get("/sessions/x2").json()["state"] == "unknown"
    ev("heartbeat", sid="x2")
    assert c.get("/sessions/x2").json()["state"] == "open"
    ev("submitted", sid="x2")
    assert c.get("/sessions/x2").json()["state"] == "closed"

def test_monitor_silent():
    now = time.time()
    ev("heartbeat", ts=now - 100, sid="s3", who="b@x.edu")
    ev("heartbeat", ts=now - 60, sid="s3", who="b@x.edu")
    row = [r for r in tapat.summarize() if r["email"] == "b@x.edu"][0]
    assert "Monitor silent" in row["flags"]

def test_csv_no_session():
    csv = "Timestamp,Email Address\n1,a@x.edu\n2,ghost@x.edu\n"
    r = c.post("/import", files={"file": ("r.csv", csv, "text/csv")}, follow_redirects=False)
    assert r.status_code == 303
    rows = {r["email"]: r for r in tapat.summarize()}
    assert rows["ghost@x.edu"]["flags"] == ["No session"]
    assert "No session" not in rows["a@x.edu"]["flags"]
```

Code listing: Chrome extension

10 files.

extension/manifest.json · JSON · 47 lines

```json
{
  "manifest_version": 3,
  "name": "Tapat",
  "version": "1.0.0",
  "description": "Exam monitor for Google Forms quizzes: blocks a listed set of sites and logs leaving the exam.",
  "permissions": [
    "declarativeNetRequest",
    "webNavigation",
    "tabs",
    "storage",
    "alarms"
  ],
  "host_permissions": [
    "https://docs.google.com/*",
    "http://localhost:8000/*",
    "<all_urls>"
  ],
  "background": {
    "service_worker": "background.js"
  },
  "action": {
    "default_popup": "popup.html",
    "default_title": "Tapat",
    "default_icon": {
      "16": "icons/icon-16.png",
      "32": "icons/icon-32.png",
      "48": "icons/icon-48.png",
      "128": "icons/icon-128.png"
    }
  },
  "web_accessible_resources": [
    {
      "resources": [
        "blocked.html"
      ],
      "matches": [
        "<all_urls>"
      ]
    }
  ],
  "icons": {
    "16": "icons/icon-16.png",
    "32": "icons/icon-32.png",
    "48": "icons/icon-48.png",
    "128": "icons/icon-128.png"
  }
}
```

extension/config.js · JavaScript · 12 lines

```
// Shared settings for the Tapat extension.
const TAPAT_SERVER = "http://localhost:8000";
const HEARTBEAT_SECONDS = 10;
// Sites blocked while an exam session is active (regex on the full URL).
const BLOCK_LIST = [
  { label: "chatgpt.com", regex: "^https?://([a-z0-9-]+\\.)*(chatgpt\\.com|chat\\.openai\\.com)/" },
  { label: "gemini.google.com", regex: "^https?://gemini\\.google\\.com/" },
  { label: "claude.ai", regex: "^https?://([a-z0-9-]+\\.)*claude\\.ai/" },
  { label: "Word Online / office.com", regex: "^https?://([a-z0-9-]+\\.)*(office\\.com|officeapps\\.live\\.com)/" },
  { label: "Google Docs", regex: "^https?://docs\\.google\\.com/document/" },
  { label: "PDF file", regex: "^[^?#]*\\.pdf([?#].*)?$" }
];
```

extension/background.js · JavaScript · 95 lines

```
// Tapat service worker: blocking, focus logging, submit detection, heartbeat.
importScripts("config.js");

async function getSession() {
  const { session } = await chrome.storage.local.get("session");
  return session || null;
}

async function send(type, detail = "") {
  const s = await getSession();
  if (!s || !s.active) return;
  const body = { session_id: s.id, student: s.email, type, detail, ts: Date.now() / 1000 };
  try {
    await fetch(`${TAPAT_SERVER}/events`, {
      method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify(body)
    });
  } catch (e) {
    // Server unreachable: the missing heartbeats show up as "Monitor silent".
  }
}

async function enableBlocking() {
  const blockPage = chrome.runtime.getURL("blocked.html");
  const rules = BLOCK_LIST.map((b, i) => ({
    id: i + 1,
    priority: 1,
    action: { type: "redirect", redirect: { regexSubstitution: `${blockPage}#\\0` } },
    condition: { regexFilter: b.regex, resourceTypes: ["main_frame", "sub_frame"] }
  }));
  const old = await chrome.declarativeNetRequest.getDynamicRules();
  await chrome.declarativeNetRequest.updateDynamicRules({
    removeRuleIds: old.map(r => r.id), addRules: rules
  });
}

async function disableBlocking() {
  const old = await chrome.declarativeNetRequest.getDynamicRules();
  await chrome.declarativeNetRequest.updateDynamicRules({ removeRuleIds: old.map(r => r.id) });
}

async function startSession(email, formUrl) {
  const session = {
    id: crypto.randomUUID(), email: email.trim().toLowerCase(), formUrl, active: true, examTabId: null
  };
  await chrome.storage.local.set({ session });
  await enableBlocking();
  chrome.alarms.create("heartbeat", { periodInMinutes: HEARTBEAT_SECONDS / 60 });
  await send("heartbeat", "session started");
  return session;
}

async function endSession(reason) {
  await send("submitted", reason);
  const s = await getSession();
  if (s) await chrome.storage.local.set({ session: { ...s, active: false } });
```

```
56    chrome.alarms.clear("heartbeat");
57    await disableBlocking();
58  }
59
60  chrome.runtime.onMessage.addListener((msg, sender, reply) => {
61    if (msg.kind === "start") {
62      startSession(msg.email, msg.formUrl).then(reply);
63      return true;
64    }
65    if (msg.kind === "examTab") {
66      getSession().then(s => s && chrome.storage.local.set({ session: { ...s, examTabI
d: sender.tab.id } }));
67    }
68    if (msg.kind === "blocked") send("blocked_url", msg.url);
69    if (msg.kind === "status") { getSession().then(reply); return true; }
70  });
71
72  chrome.alarms.onAlarm.addListener(a => { if (a.name === "heartbeat") send("heartbea
t"); });
73
74  // Chrome itself lost focus (another app or window in front).
75  chrome.windows.onFocusChanged.addListener(async (windowId) => {
76    if (windowId === chrome.windows.WINDOW_ID_NONE) send("focus_lost", "Chrome lost fo
cus");
77  });
78
79  // Leaving the exam tab.
80  chrome.tabs.onActivated.addListener(async ({ tabId }) => {
81    const s = await getSession();
82    if (s && s.active && s.examTabId !== null && tabId !== s.examTabId) {
83      const tab = await chrome.tabs.get(tabId).catch(() => null);
84      send("tab_switch", tab && tab.url ? new URL(tab.url).hostname : "other tab");
85    }
86  });
87  chrome.tabs.onCreated.addListener(() => send("tab_switch", "new tab"));
88
89  // The embedded Form frame lands on .../formResponse after submit.
90  chrome.webNavigation.onCompleted.addListener(async (d) => {
91    const s = await getSession();
92    if (s && s.active && d.tabId === s.examTabId && d.url.includes("formResponse")) {
93      endSession("formResponse");
94    }
95  }, { url: [{ hostEquals: "docs.google.com" }] });
```

extension/popup.html · HTML · 10 lines

```
<!doctype html><html><head><meta charset="utf-8"><link rel="stylesheet" href="style.css"></head>
<body class="popup">
  <h1><img src="icons/icon-32.png" alt="" style="height:24px;vertical-align:-4px;margin-right:6px">Tapat</h1>
  <p class="muted">Start your quiz here.</p>
  <label>School email<input id="email" type="email" placeholder="you@school.edu"></label>
  <label>Google Form link<input id="form" type="url" placeholder="https://docs.google.com/forms/d/e/.../viewform"></label>
  <button id="go">Open exam</button>
  <p id="err" class="err"></p>
  <script src="popup.js"></script>
</body></html>
```

extension/popup.js · JavaScript · 10 lines

```
document.getElementById("go").onclick = () => {
  const email = document.getElementById("email").value.trim();
  const form = document.getElementById("form").value.trim();
  const err = document.getElementById("err");
  if (!/^\S+@\S+\.\S+$/.test(email)) return (err.textContent = "Enter your school email.");
  if (!form.startsWith("https://docs.google.com/forms/")) return (err.textContent = "Paste the Google Form link.");
  const url = chrome.runtime.getURL(`exam.html?email=${encodeURIComponent(email)}&form=${encodeURIComponent(form)}`);
  chrome.tabs.create({ url });
  window.close();
};
```

extension/exam.html · HTML · 12 lines

```html
<!doctype html><html><head><meta charset="utf-8"><title>Tapat exam</title><link rel="stylesheet" href="style.css"></head>
<body>
  <header class="top"><b><img src="icons/icon-32.png" alt="" style="height:18px;vertical-align:-4px;margin-right:6px">Tapat</b><span id="who"></span></header>
  <section id="consent" class="card">
    <h2>Before you start</h2>
    <p class="big">This logs tab switches and apps opened, nothing else.</p>
    <p>While the quiz is open, a listed set of sites is blocked. Your teacher sees a timeline of when you left the exam. Logs are deleted after grading.</p>
    <button id="agree">I understand, start the quiz</button>
  </section>
  <iframe id="form" hidden></iframe>
  <script src="exam.js"></script>
</body></html>
```

extension/exam.js · JavaScript · 20 lines

```
const params = new URLSearchParams(location.search);
const email = params.get("email");
let form = params.get("form") || "";
document.getElementById("who").textContent = email;

function embedUrl(u) {
  const url = new URL(u);
  url.searchParams.set("embedded", "true");
  return url.toString();
}

document.getElementById("agree").onclick = async () => {
  const session = await chrome.runtime.sendMessage({ kind: "start", email, formUrl: form });
  document.getElementById("who").textContent = `${email} · session ${session.id}`;
  chrome.runtime.sendMessage({ kind: "examTab" });
  document.getElementById("consent").hidden = true;
  const frame = document.getElementById("form");
  frame.src = embedUrl(form);
  frame.hidden = false;
};
```

extension/blocked.html · HTML · 10 lines

```
<!doctype html><html><head><meta charset="utf-8"><title>Blocked by Tapat</title><link rel="stylesheet" href="style.css"></head>
<body class="blocked">
  <div class="card">
    <h1>Blocked during your quiz</h1>
    <p id="site" class="mono"></p>
    <p>This site is on your teacher's block list while the exam is open. The attempt was logged.</p>
    <button id="back">Back to the exam</button>
  </div>
  <script src="blocked.js"></script>
</body></html>
```

extension/blocked.js · JavaScript · 10 lines

```
const target = decodeURIComponent(location.hash.slice(1));
let host = target;
try { host = new URL(target).hostname + new URL(target).pathname.slice(0, 40); } cat
ch (e) {}
document.getElementById("site").textContent = host || "blocked site";
chrome.runtime.sendMessage({ kind: "blocked", url: host });
document.getElementById("back").onclick = async () => {
  const s = await chrome.runtime.sendMessage({ kind: "status" });
  if (s && s.examTabId) chrome.tabs.update(s.examTabId, { active: true });
  history.length > 1 ? history.back() : window.close();
};
```

extension/style.css · CSS · 14 lines

```
body{font-family:Inter,Segoe UI,Helvetica,Arial,sans-serif;margin:0;color:#111;background:#f5f7fb}
.popup{width:300px;padding:16px;background:#fff}
.popup h1{color:#1F3A8A;margin:0 0 4px}
label{display:block;font-size:13px;margin:10px 0}
input{display:block;width:100%;box-sizing:border-box;padding:8px;margin-top:4px;border:1px solid #ccd;border-radius:6px}
button{background:#1F3A8A;color:#fff;border:0;border-radius:6px;padding:10px 16px;font-size:14px;cursor:pointer}
.muted{color:#666;font-size:13px}.err{color:#b91c1c;font-size:13px}
.top{background:#1F3A8A;color:#fff;padding:12px 24px;display:flex;justify-content:space-between}
.card{max-width:560px;margin:60px auto;background:#fff;padding:28px;border-radius:10px;box-shadow:0 2px 8px #0002}
.card h2,.card h1{color:#1F3A8A;margin-top:0}
.big{font-size:18px;font-weight:600}
iframe{width:100%;height:calc(100vh - 46px);border:0;display:block}
.blocked{background:#fff1f1}.blocked h1{color:#9b1c1c!important}
.mono{font-family:Consolas,monospace;background:#f3f3f3;padding:6px 10px;border-radius:4px}
```

Code listing: Desktop watcher

3 files.

watcher/watcher.py · Python · 89 lines

```python
"""Tapat desktop watcher: logs which watched apps come to the front.

Only app names are sent, never window titles. Exits when the exam
session is closed on the server (the student submitted the Form).
"""
import argparse
import json
import platform
import subprocess
import sys
import time
import urllib.request

DEFAULT_APPS = ["Microsoft Word", "WINWORD", "Notes", "Notepad", "Preview", "Acroba
t", "AcroRd32"]


def frontmost_app() -> str | None:
    system = platform.system()
    if system == "Darwin":
        out = subprocess.run(
            ["osascript", "-e", 'tell application "System Events" to get name of fir
st process whose frontmost is true'],
            capture_output=True, text=True,
        )
        return out.stdout.strip() or None
    if system == "Windows":
        import psutil
        import win32gui
        import win32process
        hwnd = win32gui.GetForegroundWindow()
        _, pid = win32process.GetWindowThreadProcessId(hwnd)
        try:
            return psutil.Process(pid).name().removesuffix(".exe")
        except psutil.Error:
            return None
    return None


def matches(app: str, watched: list[str]) -> str | None:
    for name in watched:
        if name.lower() in app.lower():
            return name
    return None


def post(server, body):
    req = urllib.request.Request(f"{server}/events", data=json.dumps(body).encode(),
                                 headers={"Content-Type": "application/json"})
    urllib.request.urlopen(req, timeout=3).read()


def session_state(server, session_id) -> str:
    with urllib.request.urlopen(f"{server}/sessions/{session_id}", timeout=3) as r:
        return json.load(r)["state"]


def main(argv=None):
    p = argparse.ArgumentParser(description="Tapat desktop watcher")
    p.add_argument("--email", required=True)
```

```
    p.add_argument("--session", required=True, help="session id shown on the exam page")
    p.add_argument("--server", default="http://localhost:8000")
    p.add_argument("--apps", default=",".join(DEFAULT_APPS), help="comma-separated app names to watch")
    a = p.parse_args(argv)
    watched = [x.strip() for x in a.apps.split(",") if x.strip()]
    last = None
    tick = 0
    print(f"Tapat watcher running for {a.email}. Watching: {', '.join(watched)}")
    while True:
        app = frontmost_app()
        hit = matches(app, watched) if app else None
        if hit and hit != last:
            try:
                post(a.server, {"session_id": a.session, "student": a.email, "type": "app_opened",
                                "detail": hit, "ts": time.time()})
            except OSError:
                pass
        last = hit
        tick += 1
        if tick % 5 == 0:
            try:
                if session_state(a.server, a.session) == "closed":
                    print("Session closed. Watcher exiting.")
                    return 0
            except OSError:
                pass
        time.sleep(1)


if __name__ == "__main__":
    sys.exit(main())
```

watcher/requirements.txt · Text · 2 lines

```
psutil; sys_platform == "win32"
pywin32; sys_platform == "win32"
```

watcher/README.md · Markdown · 12 lines

```
# Tapat watcher

Run during the quiz so Word, Notes, Preview and Acrobat opening gets logged.

```
python watcher.py --email you@school.edu --session <session id from the exam page>
```

- macOS uses `osascript`; Windows needs `pip install -r requirements.txt`.
- Logs app names only, never window titles.
- Exits by itself once the Form is submitted.
- Change the list with `--apps "Microsoft Word,Notes"`.
```

Code listing: Documentation build

1 files.

docs/build_pdf.py · Python · 157 lines

```python
"""Build docs/Tapat_INTE301_Project1_Lazaro.pdf from the README, screenshots and code.

Usage: python docs/build_pdf.py   (needs markdown, pygments, playwright, pypdf)
"""
import base64
import datetime
import html
import pathlib
import re

import markdown
from pygments import highlight
from pygments.formatters import HtmlFormatter
from pygments.lexers import TextLexer, get_lexer_for_filename
from pygments.util import ClassNotFound
from playwright.sync_api import sync_playwright
from pypdf import PdfReader, PdfWriter

ROOT = pathlib.Path(__file__).resolve().parent.parent
OUT = ROOT / "docs" / "Tapat_INTE301_Project1_Lazaro.pdf"
SHOTS = ROOT / "docs" / "screenshots"
ACCENT = "#1F3A8A"
FIGURES = [
    ("01_dashboard.png", "Teacher dashboard with seed data: a clean student, a flagged student with an open timeline, and a No session row."),
    ("02_popup.png", "Extension popup where the student enters their email and the Form link."),
    ("03_consent.png", "Consent screen shown before the quiz starts."),
    ("04_exam.png", "Exam page with a sample quiz embedded in place of a real Google Form."),
    ("05_blocked.png", "Block page shown when chatgpt.com is opened during the quiz; the attempt is logged."),
    ("06_submitted.png", "Sample quiz submitted: Tapat detects the formResponse page and closes the session."),
    ("07_no_session.png", "Submitted in incognito without Tapat, flagged after CSV import."),
]
PARTS = [
    ("Teacher server", ["server/app.py", "server/seed.py", "server/run.sh", "server/requirements.txt",
                        "server/sample_responses.csv", "server/tests/test_app.py"]),
    ("Chrome extension", ["extension/manifest.json", "extension/config.js", "extension/background.js",
                          "extension/popup.html", "extension/popup.js", "extension/exam.html",
                          "extension/exam.js", "extension/blocked.html", "extension/blocked.js",
                          "extension/style.css"]),
    ("Desktop watcher", ["watcher/watcher.py", "watcher/requirements.txt", "watcher/README.md"]),
    ("Documentation build", ["docs/build_pdf.py"]),
]
LANG = {".py": "Python", ".js": "JavaScript", ".json": "JSON", ".html": "HTML", ".css": "CSS",
        ".sh": "Bash", ".txt": "Text", ".csv": "CSV", ".md": "Markdown"}

CSS = f"""
@page {{ size: A4; margin: 20mm 20mm 22mm 20mm; }}
```

```
  47  body {{ font-family: 'Inter','Helvetica Neue',Arial,sans-serif; font-size: 10.5pt;
line-height: 1.5; color: #111; }}
  48  h1, h2, h3 {{ color: {ACCENT}; break-after: avoid; }}
  49  a {{ color: {ACCENT}; }}
  50  code, pre {{ font-family: 'DejaVu Sans Mono',Consolas,monospace; font-size: 8.5pt;
}}
  51  pre {{ white-space: pre-wrap; word-break: break-all; background: #f5f7fb; padding:
8px; border-radius: 4px; }}
  52  table {{ border-collapse: collapse; width: 100%; margin: 8px 0; }}
  53  th, td {{ border: 1px solid #d5dbea; padding: 5px 7px; text-align: left; vertical-a
lign: top; }}
  54  th {{ background: #eef2fb; color: {ACCENT}; }}
  55  .page {{ break-before: page; }}
  56  .cover {{ text-align: center; padding-top: 18mm; }}
  57  .cover h1 {{ font-size: 44pt; margin: 0; }}
  58  .cover .one {{ font-size: 14pt; color: #333; margin: 6px 0 22px; }}
  59  .cover .meta {{ font-size: 11pt; line-height: 1.8; }}
  60  .cover img.logo {{ width: 95mm; border: 0; margin: 0 0 6px; }}
  61  .cover img:not(.logo) {{ width: 100%; border: 1px solid #ccd; margin-top: 18px; }}
  62  .toc ol {{ font-size: 12pt; line-height: 2; }}
  63  .readme {{ counter-reset: h2; }}
  64  .readme h1 {{ display: none; }}
  65  .readme h2 {{ counter-increment: h2; border-bottom: 2px solid {ACCENT}; padding-bot
tom: 2px; }}
  66  .readme h2::before {{ content: counter(h2) ". "; }}
  67  figure {{ break-inside: avoid; margin: 0 0 14px; text-align: center; }}
  68  figure img {{ max-width: 100%; max-height: 105mm; border: 1px solid #ccd; }}
  69  figcaption {{ font-size: 9.5pt; color: #333; margin-top: 4px; }}
  70  .label {{ background: {ACCENT}; color: #fff; font-family: 'DejaVu Sans Mono',monosp
ace; font-size: 8.5pt; padding: 5px 8px; border-radius: 4px 4px 0 0; }}
  71  .listing table {{ border: 0; margin: 0; }}
  72  .listing td {{ border: 0; padding: 0; }}
  73  .listing .linenos {{ color: #999; padding-right: 8px; user-select: none; width: 1%;
}}
  74  .listing .linenos pre, .listing .code pre {{ margin: 0; border-radius: 0; padding:
6px; line-height: 1.35; }}
  75  .listing .linenos pre {{ background: #eef0f5; word-break: normal; white-space: pre;
}}
  76  .listing tr {{ break-inside: auto; }}
  77  .part h2 {{ font-size: 20pt; }}
  78  .listing .highlight pre {{ margin: 0; border-radius: 0 0 4px 4px; line-height: 1.3
5; }}
  79  .linenos {{ color: #999; background: #eef0f5; padding: 0 6px 0 2px; margin-right: 6
px; }}
  80  {HtmlFormatter(style="friendly").get_style_defs(".highlight")}
  81  """
  82
  83
  84  def img(name, folder=SHOTS):
  85      return "" + base64.b64encode((folder / name).read_bytes
()).decode()
  86
  87
  88  def readme_html():
  89      md = (ROOT / "README.md").read_text()
  90      md = re.sub(r"## Screenshots\n.*?(?=\n## )", "## Screenshots\n\n@@FIGURES@@\n",
md, flags=re.S)
  91      body = markdown.markdown(md, extensions=["tables", "fenced_code"])
  92      figs = "".join(
  93          f'<figure><img src="{img(f)}"><figcaption><b>Figure {i}.</b> {html.escape
(c)}</figcaption></figure>'
  94          for i, (f, c) in enumerate(FIGURES, 1)
  95      )
```

```
    body = body.replace("<p>@@FIGURES@@</p>", figs)
    sections = re.findall(r"<h2>(.*?)</h2>", body)
    return body, sections


def listing(path):
    p = ROOT / path
    src = p.read_text()
    n = src.count("\n") + (0 if src.endswith("\n") else 1)
    try:
        lexer = get_lexer_for_filename(p.name, stripnl=False)
    except ClassNotFound:
        lexer = TextLexer(stripnl=False)
    code = highlight(src, lexer, HtmlFormatter(linenos="inline", style="friendly"))
    lang = LANG.get(p.suffix, "Text")
    return f'<div class="page listing"><div class="label">{path} · {lang} · {n} lines</div>{code}</div>'


def build():
    body, sections = readme_html()
    today = datetime.date.today().strftime("%B %d, %Y").replace(" 0", " ")
    toc = "".join(f"<li>{s}</li>" for s in sections)
    toc += "<li>Code listings<ol type=a>" + "".join(f"<li>{t}</li>" for t, _ in PARTS) + "</ol></li>"
    parts = "".join(
        f'<div class="page part"><h2>Code listing: {t}</h2><p>{len(files)} files.</p></div>'
        + "".join(listing(f) for f in files)
        for t, files in PARTS
    )
    doc = f"""<!doctype html><html><head><meta charset="utf-8"><title>Tapat</title>
<meta name="author" content="Troy Lazaro"><style>{CSS}</style></head><body>
<section class="cover"><img class="logo" src="{img('logo.png', ROOT / 'logo')}" alt="Tapat">
<p class="one">An exam monitor for Google Forms quizzes that blocks a listed set of sites and logs when students leave the exam.</p>
<div class="meta"><b>INTE301 Systems Integration &amp;amp; Architecture · Project 1</b><br>
Sir Rodel Villareaz<br>Troy Lazaro<br>{today}</div>
<img src="{img('01_dashboard.png')}"></section>
<section class="page toc"><h1>Contents</h1><ol>{toc}</ol></section>
<section class="page readme">{body}</section>
{parts}
</body></html>"""
    (ROOT / "docs" / "build").mkdir(exist_ok=True)
    tmp = ROOT / "docs" / "build" / "tapat.html"
    tmp.write_text(doc)
    footer = ('<div style="width:100%;font-size:8pt;color:#555;padding:0 20mm;display:flex;'
              'justify-content:space-between;font-family:Arial"><span>Tapat · INTE301 Project 1</span>'
              '<span>Page <span class="pageNumber"></span> of <span class="totalPages"></span></span></div>')
    with sync_playwright() as pw:
        b = pw.chromium.launch()
        pg = b.new_page()
        pg.goto(tmp.as_uri())
        pg.pdf(path=str(OUT), format="A4", print_background=True, display_header_footer=True,
               header_template="<div></div>", footer_template=footer,
               margin={"top": "20mm", "bottom": "22mm", "left": "20mm", "right": "20mm"})
```

```
148          b.close()
149      w = PdfWriter(clone_from=PdfReader(str(OUT)))
150      w.add_metadata({"/Title": "Tapat", "/Author": "Troy Lazaro",
151                      "/Subject": "INTE301 Systems Integration & Architecture · Proje
ct 1"})
152      w.write(str(OUT))
153      print(f"wrote {OUT}")
154
155
156  if __name__ == "__main__":
157      build()
```